

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Analiza Arhitecturală a Microserviciului **Utilizatori**

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Introducere și Contextul Sistemului	2
1.1	Harta de Context (Context Map)	2
2	Arhitectura Internă a Microserviciului Utilizatori	2
2.1	Diagrama de Pachete (Arhitectura Hexagonală)	2
2.2	Diagrama de Clase Detaliată	3
3	Design-ul Bazei de Date (Persistența)	4
4	Modelarea Domeniului (Domain-Driven Design)	5
4.1	Aggregate Root — Clasa Utilizator	5
4.2	Value Objects și Tipizarea Puternică	5
5	Șabloane de Proiectare (Design Patterns) Utilizate	7
5.1	Șablonul Adaptor (Adapter / Wrapper)	7
5.2	Șablonul Singleton (Gestionat prin Containerul IoC)	7
6	Concluzii	7

1 Introducere și Contextul Sistemului

Proiectul curent implementează un sistem backend complex pentru o agenție de bilete de avion. Pentru a asigura scalabilitatea, mentenabilitatea și independența echipelor de dezvoltare, arhitectura globală a fost divizată în **Microservicii**, ghidate de limitele contextuale (*Bounded Contexts*) din Domain-Driven Design (DDD).

1.1 Harta de Context (Context Map)

Sistemul global este împărțit în șase contexte principale, fiecare acționând ca un microserviciu independent: *Zboruri*, *Bilete*, *Utilizatori*, *Notificări*, *Statistici* și *Export*.

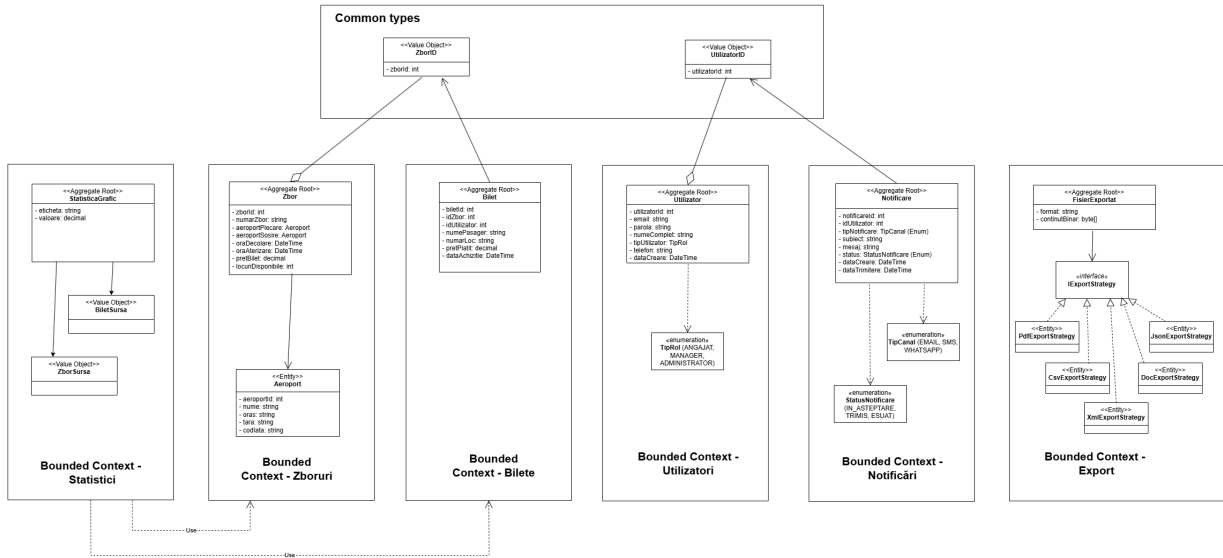


Figura 1: Harta de context a sistemului global de gestiune

Justificarea deciziei: Această decizie arhitecturală previne crearea unui monolit rigid. Contextul **Utilizatori** (vizat exclusiv în acest document) este complet izolat de detaliile operaționale ale zborurilor sau procesarea plăților biletelor. Rolul său critic este managementul identității, securizarea autentificării și autorizarea pe bază de roluri de business (*TipRol*). Tipurile de identificatori, precum `UtilizatorID`, sunt modelate ca entități imutabile (*Value Objects*) pentru a stabili granițe clare de persistență și comunicare inter-context, permițând celorlalte module să facă referire la un utilizator printr-un ID unic, fără a partaja tabele de baze de date sau detalii sensibile de securitate (cum ar fi credențialele).

2 Arhitectura Internă a Microserviciului Utilizatori

2.1 Diagrama de Pachete (Arhitectura Hexagonală)

La nivel intern, microserviciul **Utilizatori** respectă cu strictețe principiul Inversiunii Dependențelor (*Dependency Inversion Principle*), implementat printr-o Arhitectură Hexagonală curată.

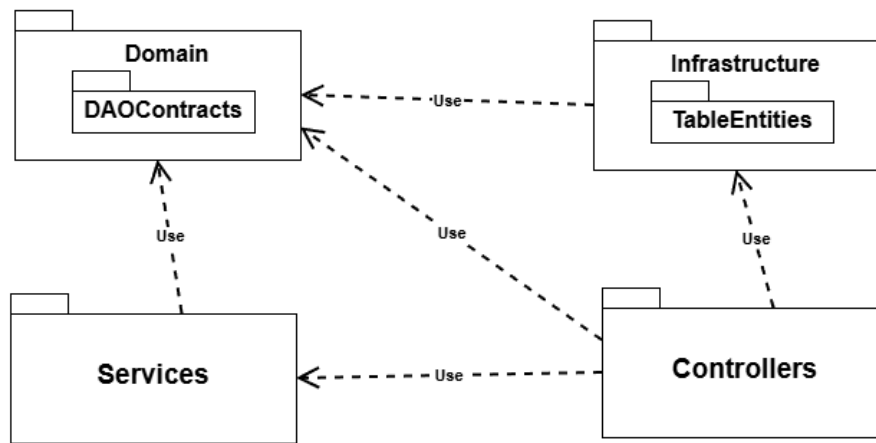


Figura 2: Diagrama de pachete: Organizarea internă pe straturi

Justificarea deciziei: Direcția dependențelor din figura 2 arată izoarea completă a pachetului **Domain**. Acesta nu importă și nu depinde de nicio componentă din framework-ul Spring sau tehnologii de persistență. Stratul de **Infrastructure** implementează interfețele decuplate din pachetul **DAOContracts** (**IUtilizatorDAO**). Stratul de **Controllers** se ocupă strict de interpretarea cererilor REST și delegarea execuției către **Services**. Serviciile interacționează exclusiv cu contractele abstracte de acces la date, permițând înlocuirea completă a bazei de date SQL sau a framework-ului ORM fără a modifica nicio linie de logică de business.

2.2 Diagrama de Clase Detaliată

Structura modulară este mapată în mod granular pentru a ilustra tipurile de date, vizibilitățile și fluxul de execuție de la recepția cererii HTTP până la stocarea fizică.

utilizator	
PK	<u>id: int</u>
	email: varchar(100) parola: varchar(100) nume_complet: varchar(100) tip_utilizator: ENUM('ANGAJAT', 'MANAGER', 'ADMINISTRATOR') telefon: varchar(20) data_creare: datetime

Figura 4: Diagrama Entitate-Relație (ERD) pentru modulul Utilizatori

Justificarea deciziei: Pentru a asigura integritatea și securitatea datelor, tabela `utilizator` definește constrângeri stricte la nivel SQL, ilustrate în figura 4. Coloanele `email` și `telefon` sunt marcate ca unice (`unique = true`), interzicând crearea de conturi duplicat. Atributul `tip_utilizator` este mapat ca text (`EnumType.STRING`) pentru a asigura trasabilitatea stărilor din Enum-ul `TipRol` direct în baza de date. Câmpul `data_creare` este configurat cu reguli de imutabilitate la actualizare (`updatable = false`, `insertable = false`) pentru a delega generarea amprente temporale exclusiv motorului bazei de date, împiedicând alterarea acesteia din exterior. Orice încălcare a constrângerilor (ex: eroare MySQL 1062 / SQLState 23000) este interceptată în controller prin intermediul unei structuri `try-catch` specializate în `DataIntegrityViolationException`, transformând excepțiile SQL brute în răspunsuri HTTP lizibile (400 Bad Request).

4 Modelarea Domeniului (Domain-Driven Design)

Nucleul de business transpune regulile operaționale ale agenției aeriene utilizând concepte tactice din Domain-Driven Design.

4.1 Aggregate Root — Clasa Utilizator

Clasa `Utilizator` acționează ca un *Aggregate Root*. Ea protejează invariantul de business și garantează starea validă a obiectului în memorie prin aplicarea următoarelor validări în constructor:

- **Validarea formatului de comunicare:** Adresa de email nu poate fi nulă și trebuie să includă obligatoriu caracterele structurale specifice ("@" și ".").
- **Strategia de fallback pentru credențiale:** Dacă parola primită este nulă sau goală, sistemul nu respinge cererea, ci aplică automat o parolă implicită securizată ("123456"), asigurând continuitatea fluxului de înregistrare rapidă.
- **Validarea prezenței datelor vitale:** Numele complet și numărul de telefon sunt attribute obligatorii (non-null și non-empty), fiind esențiale pentru emiterea ulterioară a biletelor de avion.
- **Validarea rolului de securitate:** Atributul `tipUtilizator` trebuie completat obligatoriu pentru a preveni accesul neautorizat în sistem.

4.2 Value Objects și Tipizarea Puternică

Identificatorul utilizatorului a fost extras din tipul primitiv numeric și modelat în clasa dedicată `UtilizatorID`, ca un **Value Object** imutabil. Această decizie asigură principiul de *Strong Typing*, eliminând

riscul inversării accidentale a argumentelor în semnăturile metodelor complexe din cadrul microserviciilor conexe. Clasificarea ierarhică a drepturilor de acces este administrată curat prin enumerarea `TipRol` (`ANGAJAT`, `MANAGER`, `ADMINISTRATOR`).

5 Șabloane de Proiectare (Design Patterns) Utilizate

5.1 Șablonul Adaptor (Adapter / Wrapper)

Pentru a asigura o decuplare arhitecturală completă între regulile din **Domain** și componentele din **Infrastructură**, am integrat pattern-ul structural Adaptor.

Explicație și Justificare: În contextul arhitecturii noastre evidențiate în Figura 3, interfața `IUtilizatorDAO` reprezintă **Portul** definit în interiorul domeniului pur. Clasa concretă `UtilizatorDAO` din infrastructură implementează această interfață, acționând ca un **Adaptor**. Rolul său critic este adaptarea modelelor de date: preia un obiect pur din domeniu (`Utilizator`), îl îmbracă într-o entitate recunoscută de ORM (`UtilizatorEntity`) pentru a fi procesat de Spring Data JPA, iar la citire extrage datele din tabela SQL și le reasamblează prin metoda `toDomain()` în obiecte de business libere de constrângeri tehnice. Astfel, migrarea de la un mecanism de stocare SQL la o bază de date NoSQL afectează doar codul din adaptor, lăsând logica centrală complet neatinsă.

5.2 Șablonul Singleton (Gestionat prin Containerul IoC)

Singleton este un design pattern creațional care restricționează instanțierea unei clase la o singură instanță globală la nivel de aplicație. În arhitectura curentă, managementul ciclului de viață al obiectelor este delegat containerului *Inversion of Control (IoC)* din framework-ul Spring.

Explicație și Justificare: Componentele sistemului marcate cu stereotipuri specifice Spring, precum `@RestController` (`UtilizatorController`), `@Service` (`UtilizatoriService`), `@Component` (`UtilizatorDAO`) și `@Repository` (`UtilizatorRepository`), sunt încărcate la pornire ca *Singleton Beans*.

Când mii de clienți concurenți apelează simultan API-ul de autentificare (`POST /api/utilizatori/login`), firele de execuție pornite de serverul de aplicații utilizează în mod partajat aceeași instanță unică din memorie a serviciului `UtilizatoriService`. Această decizie elimină overhead-ul alocării și colectării repetate de memorie (Garbage Collection). Serviciul și adaptorul sunt proiectate ca fiind complet *stateless* (fără stare internă modificabilă), garantând o execuție sigură și concurentă (*thread-safe*) pentru toate cererile HTTP.

6 Concluzii

Arhitectura microserviciului **Utilizatori** demonstrează o aplicare matură a principiului separării preocupărilor (*Separation of Concerns*). Prin corelarea barierelor contextuale DDD cu flexibilitatea Arhitecturii Hexagonale și eficiența șabloanelor Adaptor și Singleton, s-a obținut un modul de gestionare a utilizatorilor extrem de robust, imun la schimbările tehnologice externe și optimizat pentru testarea unitară izolată.